Practical Training of Deep Networks

Normalisation, stabilisation, and diagnosis

Jonathon Hare and Antonia Marcu

Topics

- Initialisation, normalisation, and residual pathways
- Gradient clipping, learning-rate schedules, and weight decay
- Reading training curves and inspecting internal signals
- A systematic workflow for diagnosing failed training runs

Training is a coupled dynamical system

For parameters $\boldsymbol{\theta}_t$, an optimiser produces an update

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \Delta\boldsymbol{\theta}_t, \qquad \Delta\boldsymbol{\theta}_t = \text{update}(\nabla_{\boldsymbol{\theta}}\mathcal{L}_t, \text{optimiser state}).$$

- The gradient depends on the data, architecture, current parameters, loss, and numerical precision.
- The effect of the gradient depends on the optimiser, learning rate, and parameter scale.
- A symptom such as a flat loss curve therefore has several possible causes.

The practical question

What evidence would distinguish the possible causes?

Sanity checks before a long training run

- Inspect a minibatch: shapes, dtypes, value ranges, targets, and class frequencies.
- Check the contract between the model output and the loss. For example, `CrossEntropyLoss` expects raw logits and integer class indices.
- Verify that every intended parameter receives a finite gradient.
- Disable augmentation and regularisation, then overfit one small minibatch.
  - Failure usually points to the implementation, optimisation, or data pipeline rather than generalisation.
- Compare against a simple baseline before investing in a large run.

Initialisation recap

- Random weights break symmetry between units.
- Their scale should match the activation and the layer's fan-in and fan-out.

Activation	Typical choice	Weight variance
Linear or tanh	Xavier / Glorot	$2/(n_{\text{in}} + n_{\text{out}})$
ReLU	He / Kaiming	$2/n_{\text{in}}$

- Sensible initialisation keeps early activations and gradients at a useful scale.
- Initialisation only controls the start of training; parameter updates can still move the network into a poorly scaled regime.

Normalisation controls activation scale

Suppose a layer produces pre-activations $\boldsymbol{z}$. A normalisation layer constructs

$$\hat{\boldsymbol{z}} = \frac{\boldsymbol{z} - \boldsymbol{\mu}}{\sqrt{\boldsymbol{\sigma}^2 + \epsilon}}, \qquad \boldsymbol{y} = \boldsymbol{\gamma} \odot \hat{\boldsymbol{z}} + \boldsymbol{\beta}.$$

- The choice of axes used to compute $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}^2$ defines the normalisation method.
- The learned scale $\boldsymbol{\gamma}$ and shift $\boldsymbol{\beta}$ allow the layer to recover a useful representation.
- Normalisation changes the optimisation problem. It does not merely standardise the final representation.
- The small constant ϵ prevents division by zero and affects numerical behaviour when the variance is tiny.

Batch normalisation

For feature or channel k and a minibatch containing m values:

$$\mu_k = \frac{1}{m}\sum_{i=1}^{m} z_{ik}, \qquad \sigma_k^2 = \frac{1}{m}\sum_{i=1}^{m}(z_{ik} - \mu_k)^2,$$

$$y_{ik} = \gamma_k \frac{z_{ik} - \mu_k}{\sqrt{\sigma_k^2 + \epsilon}} + \beta_k.$$

- For a fully connected layer, statistics are normally computed over the batch for each feature.
- For a convolutional layer, they are normally computed over the batch and spatial positions for each channel.
- Each example therefore influences the normalised representation of the other examples in its minibatch.

BatchNorm during training and evaluation

Training mode

- Normalises using the current minibatch statistics.
- Updates running estimates of the mean and variance.
- Output for one example depends on the other examples in the minibatch.

Evaluation mode

- Normalises using stored running statistics.
- Gives a stable result for one example independently of the evaluation batch.
- Does not update the stored statistics.

PyTorch state matters

`model.train()` and `model.eval()` change BatchNorm behaviour. They do not enable or disable gradient recording.

Batch size and data dependence

- Small or unrepresentative minibatches give noisy estimates of the activation statistics.
- Large changes in batch size can therefore change the optimisation behaviour even when the learning rate is unchanged.
- Correlated examples within a batch can bias its statistics.
  - Examples include neighbouring video frames or patches from the same image.
- Running statistics can become inaccurate when fine-tuning on a small dataset or when the deployment distribution changes.
- Freezing parameters does not automatically freeze BatchNorm statistics. The module's training or evaluation mode controls them.

Layer normalisation

For one example with d features, LayerNorm computes

$$\mu = \frac{1}{d}\sum_{j=1}^{d} z_j, \qquad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(z_j - \mu)^2.$$

- Each example supplies its own statistics, so the result does not depend on the other examples in the minibatch.
- The same computation is used during training and evaluation. There are no running statistics.
- LayerNorm is therefore convenient for variable-length sequences and small batches.

A useful distinction

BatchNorm normalises each feature or channel across examples. LayerNorm normalises features within each example.

Residual pathways preserve a direct route

A residual block computes

$$\boldsymbol{y} = \boldsymbol{x} + F(\boldsymbol{x}; \boldsymbol{\theta}), \qquad \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \boldsymbol{I} + \frac{\partial F}{\partial \boldsymbol{x}}.$$

- The identity term provides a direct route for activations and gradients.
- The residual branch learns a correction to the current representation rather than a complete replacement.
- Training is often more stable when the residual branch starts with a relatively small scale.
  - One practical method initialises the final normalisation scale in the branch to zero, so the block initially behaves approximately like the identity.
- Initialisation, normalisation, and residual structure therefore address related but distinct sources of poor signal propagation.

Gradient clipping limits unstable updates

For global norm clipping with threshold τ:

$$\tilde{\boldsymbol{g}} = \begin{cases} \boldsymbol{g}, & \|\boldsymbol{g}\|_2 \le \tau, \\ \tau \dfrac{\boldsymbol{g}}{\|\boldsymbol{g}\|_2}, & \|\boldsymbol{g}\|_2 > \tau. \end{cases}$$

- Norm clipping preserves the gradient direction while limiting its magnitude.
- Value clipping clamps individual coordinates and can change the direction substantially.
- Apply clipping after `backward()` and before `step()`:

Learning-rate schedules and warm-up

- The learning rate controls the scale of the parameter update, rather than the size of the gradient itself.
- Warm-up starts with a small learning rate and increases it over the first part of training.
  - This limits early updates while activations, gradients, and optimiser state are still settling.
- Decay then reduces the learning rate as training approaches a useful solution.
- Record the learning rate alongside the loss. A scheduler called at the wrong frequency can otherwise be difficult to diagnose.

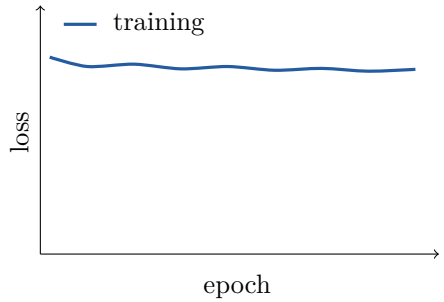
Weight decay and optimisation

Decoupled weight decay applies a direct shrinkage to the parameters:

$$\boldsymbol{\theta}_{t+1} = (1 - \eta_t \lambda)\boldsymbol{\theta}_t - \eta_t \text{ update}(\boldsymbol{g}_t).$$

- For plain SGD, adding an L2 penalty to the loss is equivalent.
- Adaptive optimisers rescale gradient coordinates, so an L2 gradient penalty and direct weight decay need not have the same effect.
- AdamW makes the decay explicit and separate from the adaptive gradient update.
- Biases and normalisation scale and shift parameters are often excluded from decay, depending on the training recipe.
- Weight decay can help, but it does not guarantee good generalisation.

Learn to read the training curves

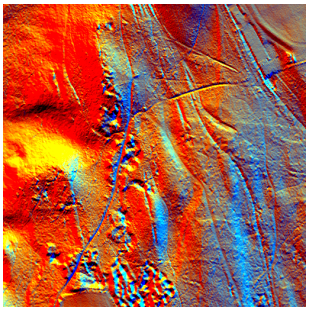


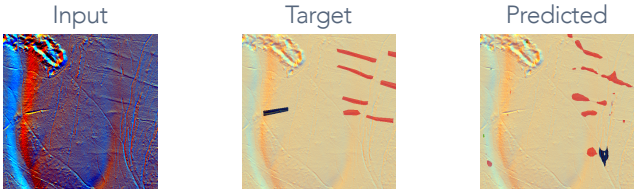
- Is this underfitting, or has optimisation failed?
- What could prevent the loss from falling?
- What would you inspect before changing the architecture?
- How would the one-minibatch test help?

Curves provide evidence

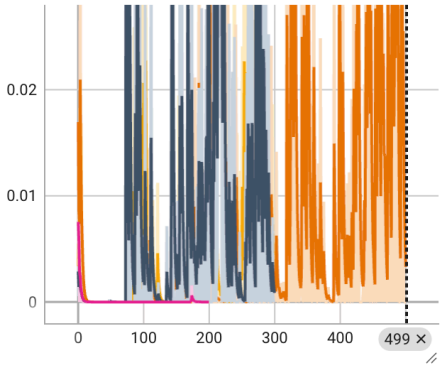
Use them to propose the next measurement or design a controlled experiment.

Identify Archaeological Artefacts



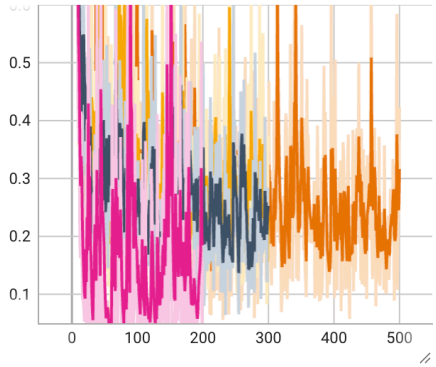


epoch/IoU

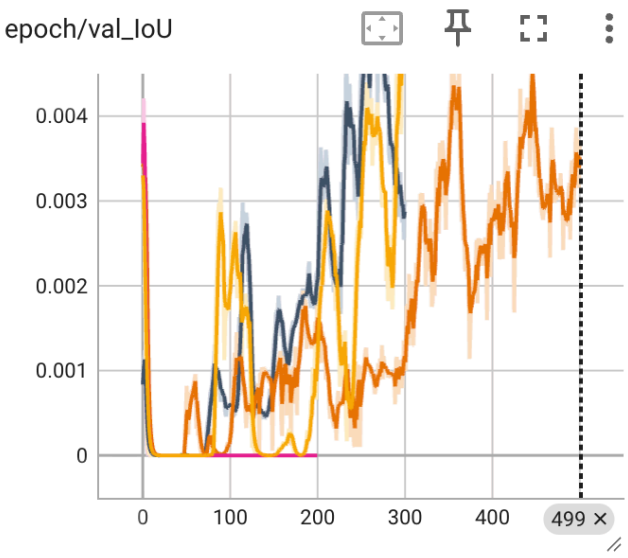


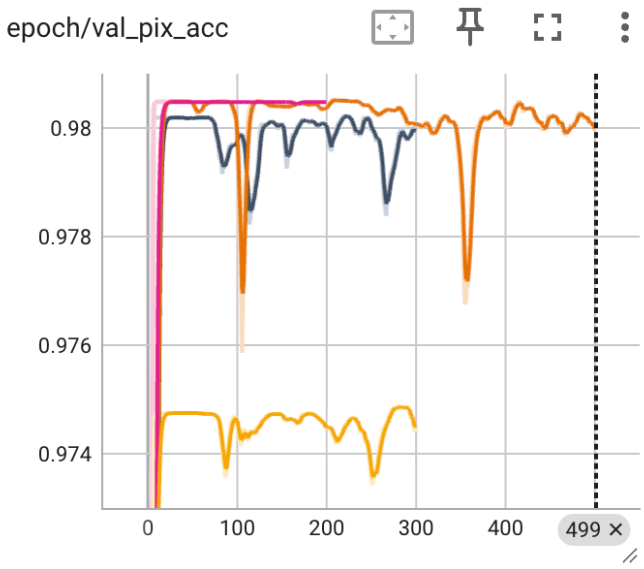
Run	Smoothed	Value
logs/testing/FCN_Mar04_17-57-17-run-0	0	0
logs/testing/FCN_Mar04_19-13-59-run-0	0.0009	0

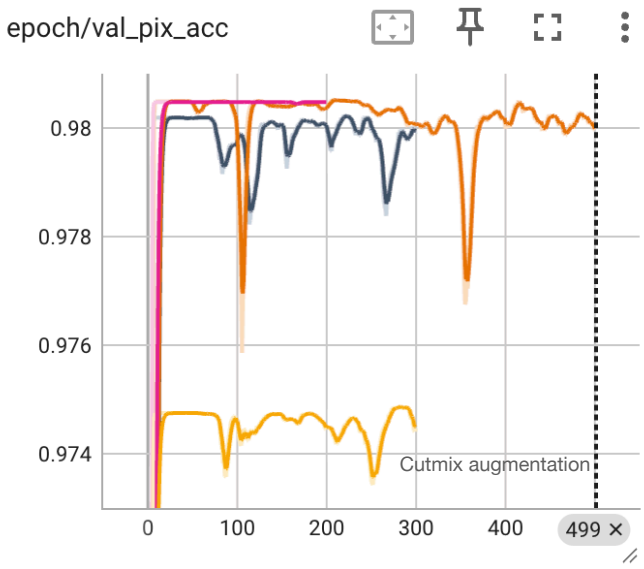
epoch/loss

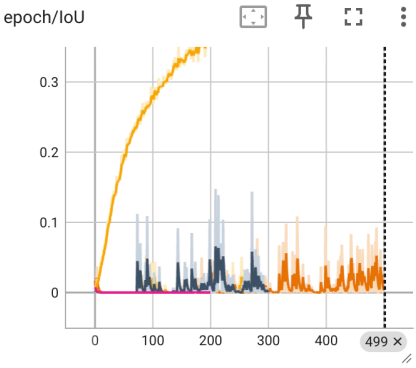


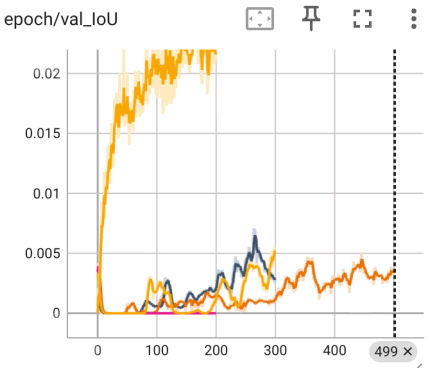
Run	Smoothed	Value
logs/testing/FCN_Mar04_17-57-17-run-0	0.1844	0.1575
logs/testing/FCN_Mar04_19-13-59-run-0	0.2527	0.2783

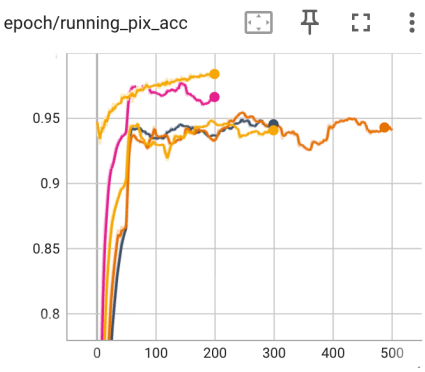

epoch/val_IoU
0.004
0.003
0.002
0.001
0
0
100
200
300
400
499

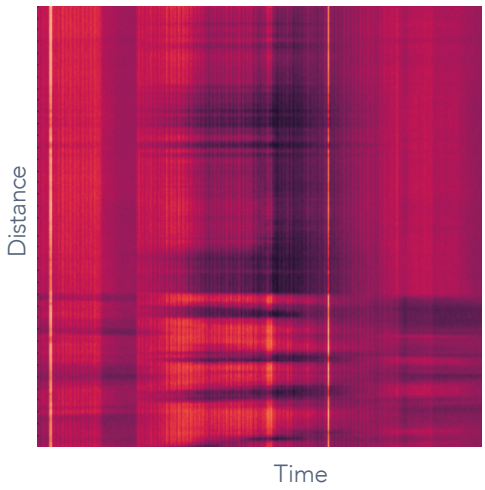

epoch/val_pix_acc
0.98
0.978
0.976
0.974
0
100
200
300
400
499

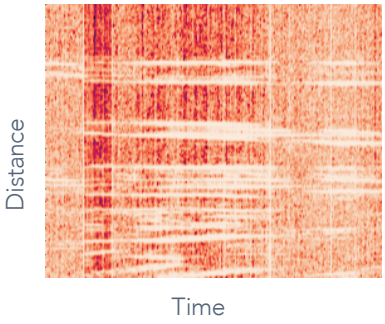

epoch/val_pix_acc
0.98
0.978
0.976
0.974
Cutmix augmentation
0
100
200
300
400
499

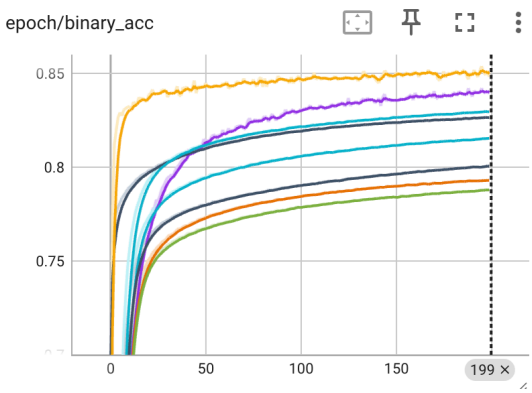

epoch/IoU
0.3
0.2
0.1
0
0
100
200
300
400
499

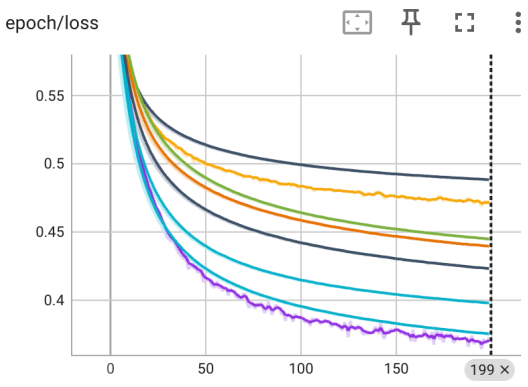

epoch/val_IoU
0.02
0.015
0.01
0.005
0
0
100
200
300
400
499

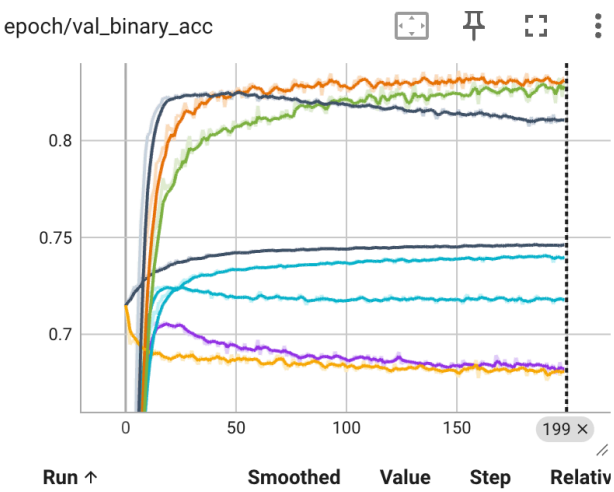

epoch/running_pix_acc
0.95
0.9
0.85
0.8
0
100
200
300
400
500

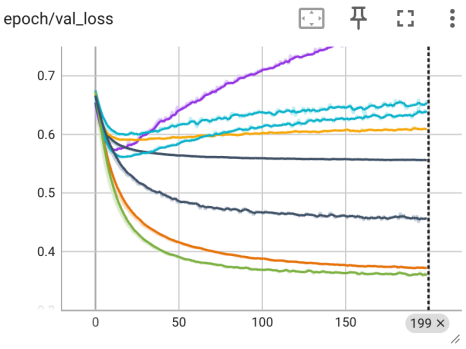

Distance
Time

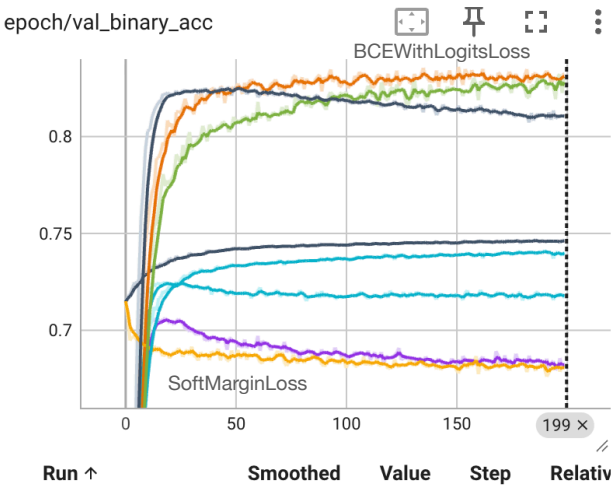

Distance
Time

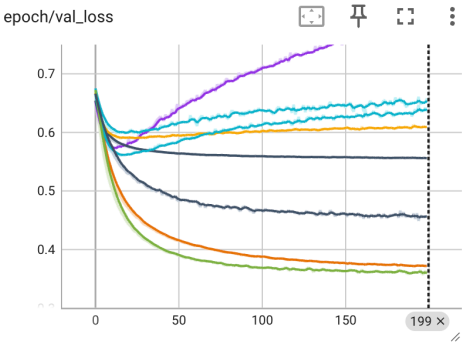

epoch/binary_acc
0.85
0.8
0.75
0
50
100
150
199 ×


epoch/loss
0.55
0.5
0.45
0.4
0
50
100
150
199 ×

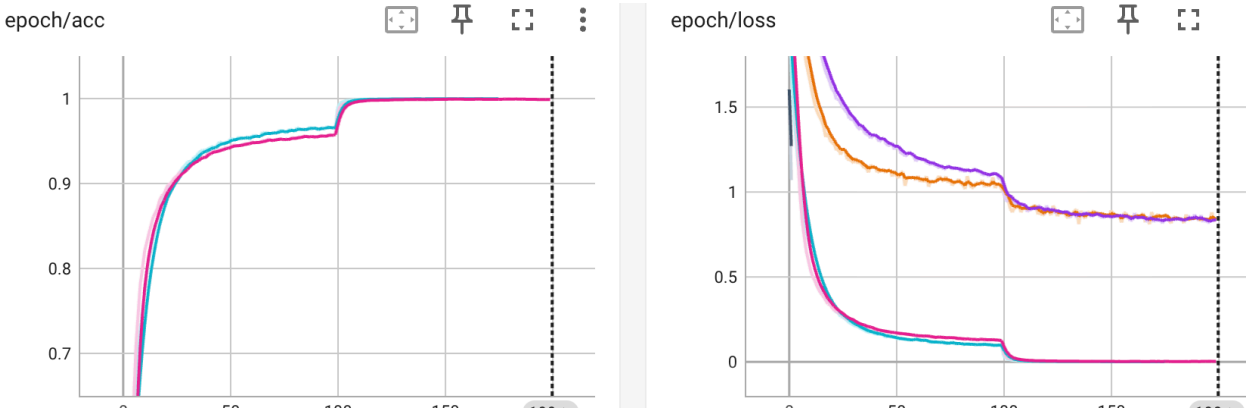

epoch/val_binary_acc
0.8
0.75
0.7
0
50
100
150
199
Run
Smoothed
Value
Step
Relativ

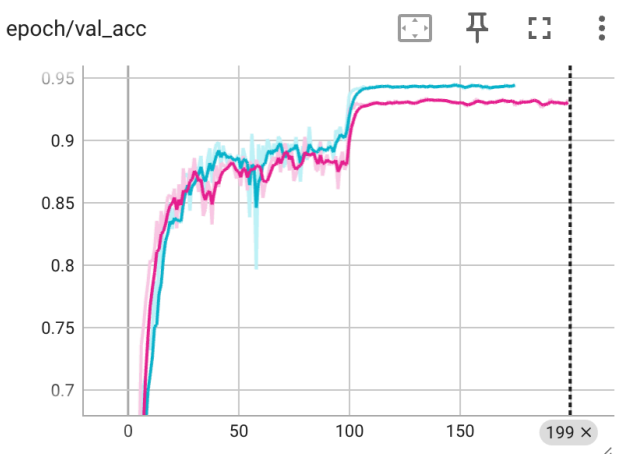

epoch/val_loss
0.7
0.6
0.5
0.4
0
50
100
150
199

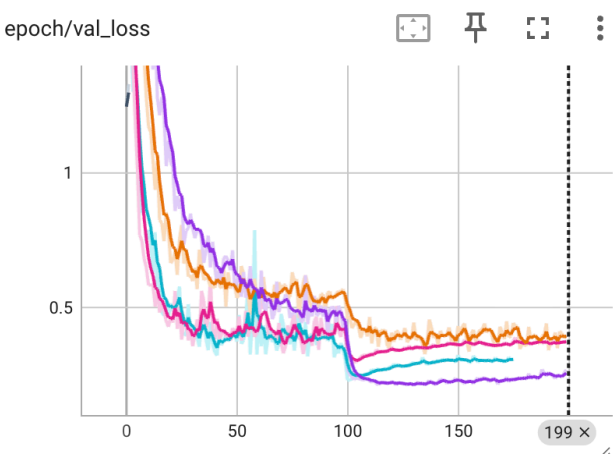

epoch/val_binary_acc
BCEWithLogitsLoss
0.8
0.75
0.7
SoftMarginLoss
0
50
100
150
199 ×
Run ↑
Smoothed
Value
Step
Relativ


epoch/val_loss
0.7
0.6
0.5
0.4
0
50
100
150
199 ×

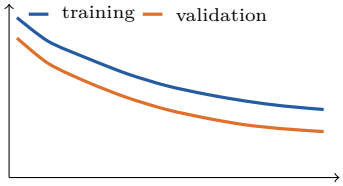
Back to C10



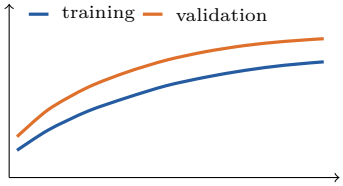

epoch/val_acc
0.95
0.9
0.85
0.8
0.75
0.7
0
50
100
150
199


epoch/val_loss
1
0.5
0
50
100
150
199

Validation looks better than training



Loss[0.2em]



Accuracy[0.2em]

- Does this necessarily indicate data leakage?
- How could augmentation, dropout, or the handling of BatchNorm contribute?
- Were the training and validation metrics computed in directly comparable ways?

Inspecting activations, gradients, and updates

Useful summaries for each layer include:

- activation mean, standard deviation, range, and fraction of exact zeros;
- gradient norm and the fraction of non-finite values;
- parameter norm and the relative update
$$\frac{\|\Delta \boldsymbol{\theta}\|_2}{\|\boldsymbol{\theta}\|_2 + \epsilon};$$
- how these quantities change over training, rather than only their value at one step.
- PyTorch has Forward and backward hooks which can be used to collect these summaries without changing the model computation.

A practical debugging sequence

1. **Reproduce the failure.** Keep the data, configuration, and logs needed to see it again.
2. **Check the inputs and objective.** Inspect a batch and verify the model-loss compatibility.
3. **Try to overfit one small batch.**
    - If this fails, simplify the model and inspect its activations, gradients, and parameter updates.
    - If this works, restore the full training pipeline one component at a time.
4. **Find the first damaging signal.** Identify the layer or training step where values vanish, explode, or become NaN or inf.
5. **Separate training from generalisation.** If training succeeds but validation does not, investigate the data split, distribution, pre-processing and regularisation.

Working method

Change one factor at a time and return to a known working baseline after an unsuccessful change.